Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas







Analizador Léxico y Sintáctico de Comandos EXT2

PONDERACIÓN: 20 pts

Tiempo estimado: 35 hrs/min

Índice

1. MARCO FORMATIVO....3
1.1. Valor....3
1.2. Competencia(s)....3
1.3. Habilidad(es) blandas a formar....3
2. Resultado del Aprendizaje....3
2.1. Objetivo SMART....3
3. Enunciado de la Práctica....4
3.1 Descripción del problema a resolver....4
3.2 Alcance de la práctica....4
4. Entregables....2
5. Material de apoyo....3
6. Recursos y herramientas a utilizar....3
7. Cronograma....4
8. Rúbrica de Calificación....4
8.1 Requisitos para optar a la calificación....4
8.2 Resumen de Puntuaciones....5
Detalle de la Calificación....6
Comentarios Generales....8

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	La responsabilidad se aplica al validar correctamente la estructura de los comandos ingresados, identificando errores léxicos y sintácticos sin ejecutar instrucciones que alteren archivos o particiones.

1.2. Competencia(s)

El estudiante adquirirá la capacidad de construir herramientas de análisis de lenguajes aplicadas a la administración de sistemas de archivos

1.3. Habilidad(es) blandas a formar

Se fomentará la capacidad analítica para la resolución de problemas técnicos complejos y la atención al detalle en la validación de sintaxis.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	**Medible (¿Cuánto?)**	**Alcanzable (¿Cómo?)**	**Realista (¿Para qué?)**	**A Tiempo (¿Cuándo?)**
Desarrollar un analizador léxico y sintáctico para comandos EXT2.	Validar correctamente los comandos asignados	Integrando un backend en C++ con un frontend en React/Angular/ Vue mediante una API REST.	Para detectar errores de gramática y parámetros sin alterar archivos físicos.	Completando el desarrollo entre el 27/07/2026 y el 11/08/2026.

3. Enunciado de la Práctica

3.1 Descripción del problema a resolver

Se necesita desarrollar una aplicación web capaz de recibir comandos relacionados con la administración de discos y sistemas de archivos EXT2. El sistema únicamente deberá analizar la estructura léxica y sintáctica de los comandos ingresados, detectando errores de escritura, parámetros inválidos y problemas de gramática, sin ejecutar ninguna acción real sobre archivos o particiones. Para ello, se crea una solución que integra un backend en C++ y un frontend interactivo. En el backend, el sistema de archivos se gestionará en una distribución Linux, que atenderá todas las solicitudes provenientes del frontend.

3.2 Alcance de la práctica

La práctica incluye un frontend con área de entrada y salida de comandos, además de un backend en C++ encargado del análisis léxico y sintáctico.

Los comandos a analizar serán únicamente:

- MKDISK
- RMDISK
- FDISK
- MOUNT
- MKFS
- MKUSR
- RMUSR
- MKFILE

El sistema NO debe ejecutar comandos ni modificar archivos físicos.

COMANDOS

1. MKDISK

Este comando creará un archivo binario que simulará un disco

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-size	**Obligatorio**	Este parámetro recibirá un número que indicará el tamaño del disco a crear. Debe ser positivo y mayor que cero, si no se mostrará un error.
-fit	**Opcional**	Indicará el ajuste que utilizará el disco para crear las particiones dentro del disco Podrá tener los siguientes valores: **BF**: Indicará el mejor ajuste (Best Fit) **FF**: Utilizará el primer ajuste (First Fit) **WF**: Utilizará el peor ajuste (Worst Fit) Ya que es opcional, se tomará el primer ajuste (FF) si no está especificado en el comando. Si se utiliza otro valor que no sea alguno de los anteriores mostrará un mensaje de error.
-unit	**Opcional**	Este parámetro recibirá una letra que indicará las unidades que utilizará el parámetro size. Podrá tener los siguientes valores: **K:** Indicará que se utilizarán Kilobytes (1024 bytes) **M:** Indicará que se utilizarán Megabytes (1024 * 1024 bytes) Este parámetro es opcional, si no se encuentra se creará un disco con tamaño en Megabytes. Si se utiliza otro valor debe mostrarse un mensaje de error.
-path	**Obligatorio**	Este parámetro será la ruta en el que se creará el archivo que representará el disco duro

Ejemplos:

```
#Crea un disco de 3000 Kb en la carpeta home
mkdisk -Size=3000 -unit=K -path=/home/user/Disco1.mia
```

```
#No es necesario utilizar comillas para la ruta en este caso ya que
la ruta no tiene ningún espacio en blanco
mkdisk -path=/home/user/Disco2.mia -Unit=K -size=3000

#Se ponen comillas por la carpeta "mis discos" ya que tiene
espacios en blanco, se crea si no está no existe
mkdisk -size=5 -unit=M -path="/home/mis discos/Disco3.mia"

#Creará un disco de 10 Mb ya que no hay parámetro unit
mkdisk -size=10 -path="/home/mis discos/Disco5.mia"
```

2. RMDISK

Este parámetro elimina un archivo que representa a un disco duro. Tendrá los siguientes parámetros

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-path	**Obligatorio**	Este parámetro será la ruta en el que se eliminará el archivo que representará el disco duro.

Ejemplo:

```
#Elimina Disco4.mia
rmdisk -path="/home/mis discos/Disco4.mia
```

3. FDISK

Este comando se encarga de la administración de particiones en el archivo que representa al disco duro virtual. Permite a los usuarios crear, eliminar o modificar particiones dentro del archivo de disco duro.

Parámetro	Categoría	Descripción
-size	**Obligatorio al crear**	Este parámetro recibirá un número que indicará el tamaño de la partición a crear. Debe ser positivo y mayor a cero, de lo contrario se mostrará un mensaje de error.
-unit	**Opcional**	Este parámetro recibirá una letra que indicará las unidades que utilizará el parámetro s. Podrá tener los siguientes valores: **B**: indicará que se utilizarán bytes. **K**: indicará que se utilizarán Kilobytes(1024 bytes) **M**:indicará que se utilizarán Megabytes(1024 * 1024 bytes). Este parámetro es opcional, si no se encuentra se creará una partición en Kilobytes. Si se utiliza un valor diferente mostrará un mensaje de error.
-path	**Obligatorio**	Este parámetro será la ruta en la que se encuentra el disco en el que se creará la partición.

-type	**Opcional**	Indicará que tipo de partición se creará. Ya que es opcional, se tomará como primaria en caso de que no se indique. Podrá tener los siguientes valores: **P**: Se creará una partición primaria. **E**: Se creará una partición extendida. **L**: Se creará una partición lógica. Si se utiliza otro valor diferente a los anteriores deberá mostrar un mensaje de error.
-fit	**Opcional**	Indicará el ajuste que utilizará la partición para asignar espacio. Podrá tener los siguientes valores: **BF**: Indicará el mejor ajuste (Best Fit) **FF**: Utilizará el primer ajuste (First Fit) **WF**: Utilizará el peor ajuste (Worst Fit)
-name	**Obligatorio**	Indicará el nombre de la partición. El nombre no debe repetirse dentro de las particiones de cada disco.

Ejemplos:

```
#Crea una partición primaria llamada Particion1 de 300kb
#con el peor ajuste en el disco Disco1.mia
fdisk -Size=300 -path=/home/Disco1.mia -name=Particion1
```

```
#Crea una partición extendida dentro de Disco2 de 300kb
#Tiene el peor ajuste
fdisk -type=E -path=/home/Disco2.mia -Unit=K -name=Particion2
-size=300


#Crea una partición lógica con el mejor ajuste, llamada Partición
3 de 1 Mb en el Disco3
fdisk -size=1 -type=L -unit=M
-fit=BF-path="/mis
discos/Disco3.mia"-name="Particion3"


#Intenta crear una partición extendida dentro de Disco2 de 200 kb
#Debería mostrar error ya que ya existe una partición extendida
#dentro de Disco2
fdisk -type=E -path=/home/Disco2.dk -name=Part3 -Unit=K -size=200
```

4. MOUNT

Este comando montará una partición del disco en el sistema. Cada partición se identificará por un id que tendrá la siguiente estructura utilizando el número de carnet:

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-path	**Obligatorio**	Este parámetro será la ruta en la que se encuentra el disco que se montará en el sistema. Este archivo ya debe existir.
-name	**Obligatorio**	Indica el nombre de la partición a cargar. Si no existe debe mostrar error.

Ejemplos:

```
#Monta las particiones de los siguientes disco
*canet = 202401234
mount   -path=/home/Disco2.mia   -name=Part2
#id=341A     mount     -path=/home/Disco2.mia
-name=Part1          #id=342A          mount
-path=/home/Disco1.mia  -name=Part3  #id=341B
mount   -path=/home/Disco3.mia   -name=Part2
#id=341C
```

5. MKFS

Este comando realiza un formateo completo de la partición, se formatea como ext2.

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-id	**Obligatorio**	Indicará el id que se generó con el comando mount.
-type	Opcional	Indicará que tipo de formateo se realizará. Podrá tener los siguientes valores: **Full**: en este caso se realizará un formateo completo. Ya que es opcional, se tomará como un formateo completo si no se especifica esta opción.

Ejemplos:

```
#Realiza un formateo completo de la partición en el id 341A
en ext2
mkfs -type=full -id=341A

#Realiza un formateo completo de la partición que ocupa el id 342A
mkfs -id=342A
```

6. MKUSR

Este comando crea un usuario en la partición. Solo lo puede ejecutar el usuario root, Recibirá los siguientes parámetros:

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-user	**Obligatorio**	Indicará el nombre del usuario a crear, **Máximo: 10 caracteres.**
-pass	**Obligatorio**	Indicará la contraseña del usuario **Máximo 10 Caracteres**
-grp	**Obligatorio**	Indicará el grupo al que pertenece el usuario. **Máximo 10 Caracteres**

Ejemplo:

```
#Crea el usuario user1
mkusr -user=user1
-pass=1234 -grp=usuarios

#Debe mostrar mensaje de error porque el parámetro
pass no existe
mkusr -user=user2 -grp=admins
```

7. RMUSR

Este comando elimina un usuario en la partición. Recibirá los siguientes parámetros:

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-user	**Obligatorio**	Indicará el nombre del usuario a eliminar

Ejemplo:

```
#Elimina el usuario user1
rmusr -user=user1

#Debe mostrar mensaje de error porque el user1 ya no
existe
rmusr -user=user1
```

8. MKFILE

Este comando permitirá crear un archivo, Tendrá los siguientes parámetros:

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-path	**Obligatorio**	Este parámetro será la ruta del archivo que se creará. Si lleva espacios en blanco deberá encerrarse entre comillas. Si ya existe debe mostrar un mensaje si se desea sobreescribir el archivo.
-r	**Opcional**	Si se utiliza este parámetro y las carpetas especificadas por el parámetro **path** no existen
-size	**Opcional**	Este parámetro indicará el tamaño en bytes del archivo, El contenido serán números del 0 al 9 cuantas veces sea necesario hasta cumplir el tamaño ingresado. Si no se utiliza este parámetro, el tamaño será 0 bytes. Si es negativo debe mostrar error.
-cont	**Opcional**	Indicará un archivo en el disco de la computadora, (entiéndase su computadora) que tendrá el contenido del archivo. Se utilizará para cargar contenido en el archivo

Ejemplos:

```
#Crea el archivo a.txt
#Si no existen las carpetas home user o docs se
crean #El tamaño del archivo es de 15 bytes
#El contenido sería:
#012345678901234
mkfile -size=15 -path=/home/user/docs/a.txt -r

#Crea "archivo 1.txt" la carpeta "mis documentos" ya debe
existir #el tamaño es de 0 bytes
mkfile -path="/home/mis documentos/archivo 1.txt"

#Crea el archivo b.txt
#El contenido del archivo será el mismo que el archivo b.txt
#que se encuentra en el disco duro de la computadora.
mkfile -path=/home/user/docs/b.txt -r -cont=/home/Documents/b.txt
```

3.3 Requerimientos técnicos

Define las tecnologías, herramientas o lenguajes de programación que los estudiantes deberán utilizar o integrar en el desarrollo de la práctica.

- Backend obligatorio en C++.
- Frontend desarrollado con React, Angular o Vue.
- Comunicación mediante API REST.
- Implementación de análisis léxico y sintáctico.
- Validación de parámetros obligatorios y opcionales.

4. Entregables

Describe los productos concretos que se espera que los estudiantes entreguen al finalizar la práctica. Pueden ser prototipos, informes técnicos, documentación, etc.

Tipo	Descripción
Sitio Web Funcional	Aplicación web ejecutable localmente que permita interactuar con el sistema de archivos simulado, ejecutar comandos y visualizar resultados.
Comandos Implementados	Implementación de la mayoría de los comandos definidos en el enunciado
Documentación Técnica	Manual que incluya la arquitectura del sistema, explicación de las estructuras implementadas y ejemplos de uso de los comandos.
Repositorio GitHub	Proyecto completo subido a un repositorio privado de GitHub, con acceso habilitado para los auxiliares, siguiendo el formato requerido.
Manual de Usuario	Documento que explica cómo usar el sistema desarrollado, incluyendo capturas de pantalla, pasos detallados y resolución de problemas comunes.

Nombre del repositorio: **MIA_2S2026_Practica_carnet**
Usuarios de github de los auxiliares de cada sección:

5. ***Sección A***: JSamRojas
6. ***Sección B***:BramlleyProgram
7. ***Sección C***: gkruiz
8. ***Sección D***: nxz7

5. Material de apoyo

Colocar material de apoyo que el estudiante puede consultar para la elaboración de la práctica. Incluye links , libros , videos. Todo lo que pueda apoyar al estudiante a lograr el desarrollo de la práctica.

- https://youtu.be/YkGpSUpkWcE

6. Recursos y herramientas a utilizar

Listado de materiales que los estudiantes deberán usar o investigar:

- **Software/Hardware:** Uso de un framework u herramienta moderna para la creación del Frontend.
- **Lenguaje de programación:** C++ para la implementación del backend y lógica del sistema de archivos.
- **Plataformas:** GitHub (para repositorio), UEDI (para entrega).
- **Lecturas recomendadas:** Manuales, lecturas.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de práctica	27/07/2026	-
Desarrollo	27/07/2026	11/08/2026
Calificación		

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación de la práctica, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	**Descripción**	**Cumple (Sí/No)**
Frontend funcional	La interfaz debe permitir ingresar comandos y visualizar errores léxicos y sintácticos.	
Backend en C++	El backend debe analizar comandos sin ejecutar operaciones reales.	
Entrega en UEDI	El estudiante realizó la entrega del link de su repositorio en el apartado de UEDI a tiempo	
LINUX	El estudiante debe presentar el proyecto en el sistema operativo LINUX - INSTALACIÓN FÍSICA, NO VM	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades	15	
Documentación técnica (Manual técnico y de usuario en PDF)	10	
Preguntas relacionadas al proyecto (dominio técnico y explicación)	5	
Sub-Total Habilidades	**15**	
2. Conocimiento		
Frontend y Backend	15	
Análisis Léxico	7	
Análisis Sintáctico	7	
Creación y Manejo del Sistema de Archivos Ext2 (MKDISK, FDISK, RMDISK, MOUNT)	28	
Comandos adicionales Ext2 (MKFS, MKUSR, RMUSR, MKFILE)	28	
Sub-Total Conocimiento	**85**	
Total	100	

*La calificación debe incluir ambas áreas conocimientos y habilidades.

Detalle de la Calificación

En este apartado se deberá usar la siguiente plantilla para la rúbrica que debe tener obligatoriamente el criterio de evaluación, el rango de calificación **Satisfactorio (100%–61%)** y el rango de necesita mejorar **(60%–0%).**

	Criterio/Nivel			**Nivel de evaluación**		
No.	**Criterio de evaluación**	**Punteo maximo**		**Satisfactorio 100% - 61%**	**Necesita mejorar 60% - 0%**	**Punteo Obtenido**
1	Habilidades					
1.1	Documentación técnica (Manual técnico y de usuario en PDF)	10	Descripción	Se debe entregar un manual técnico con la arquitectura del sistema, estructuras utilizadas, comandos implementados y **un manual de usuario con capturas**	La documentación es incompleta o confusa, no se incluye: arquitectura del sistema, estructuras utilizadas, comandos implementados y un manual de usuario con capturas	
			Rango del punteo	10-6 /	5 - 0/	
1.2	Preguntas relacionadas al proyecto (dominio técnico y explicación)	5	Descripción	Responde con precisión y coherencia, demostrando dominio del funcionamiento del sistema, comandos, estructuras y arquitectura implementada.	Respuestas superficiales o incorrectas; evidencia desconocimiento del funcionamiento, comandos, estructuras o arquitectura.	
			Rango del punteo	5 - 3 /	2 - 0/	
	Sub-Total de Puntos	**15**				0

2	Conocimientos					
2.1	Parte 1: Frontend y Backend	15	Descripción	El frontend está completo, muestra entrada y salida de comandos, y ejecuta scripts correctamente. La interfaz es funcional e intuitiva.	El frontend presenta fallos de ejecución, interfaz poco clara, o no permite cargar/ejecutar comandos.	
			Rango del punteo	15-8 /	7 - 0/	
2.2	Análisis Léxico	**7**	Descripción	Se extraen correctamente los tokens para validar que el comando ingresado es válido	No se reconocen correctamente los tokens o no muestra error en la consola	
			Rango del punteo	7-4	4-0	
2.3	Análisis Sintáctico	**7**	Descripción	Se reconoce correctamente el orden y estructura de los comandos ingresados	No se reconoce el ingreso correcto de los comandos ni la estructura de los mismo	
			Rango del punteo	7-4	4-0	
2.4	Creación y Manejo del Sistema de Archivos Ext2 (MKDISK, FDISK, RMDISK, MOUNT)	**28**	Descripción	Los comandos relacionados con MKDISK funcionan correctamente	Los comandos fallan, y no permiten el presentar un sistema de archivos básico	

2.5	Comandos adicionales Ext2 (MKFS , MKUSR, RMUSR, MKFILE)	28	Rango del punteo	28-14 /	14-0/	
			Descripción	Los comandos simulan correctamente el formateo de la partición, la creación de grupos, archivos y el logueo en consola	Los comandos presentan errores, no simulan correctamente el sistema de archivos básico	
			Rango del punteo	28-14	14-0	
	Sub-Total de Puntos	85				0
	Total	100				

Comentarios Generales

Apartado para comentar si hubo algún inconveniente durante la calificación o comentario.